Ahmed GHALEB

temp_psm2_chunks-55-68

Chapter 5 Results, Testing and Discussion (System Development Based) / Results, Analysis and Discussion (Research Based)

Document Details

Submission ID
trn:oid:::3618:144448101

Submission Date
Jun 29, 2026, 2:26 AM GMT+8

Download Date
Jun 30, 2026, 7:04 AM GMT+8

File Name
temp_psm2_chunks-55-68.pdf

File Size
1.4 MB

14 Pages

1,810 Words

10,637 Characters

10% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Filtered from the Report

- Bibliography
- Quoted Text
- Small Matches (less than 8 words)

Match Groups

17 Not Cited or Quoted 10%
Matches with neither in-text citation nor quotation marks

0 Missing Quotations 0%
Matches that are still very similar to source material

0 Missing Citation 0%
Matches that have quotation marks, but no in-text citation

0 Cited and Quoted 0%
Matches with in-text citation present, but no quotation marks

Top Sources

3% Internet sources
1% Publications
8% Submitted works (Student Papers)

Match Groups

- **17** Not Cited or Quoted 10%
  Matches with neither in-text citation nor quotation marks
- **0** Missing Quotations 0%
  Matches that are still very similar to source material
- **0** Missing Citation 0%
  Matches that have quotation marks, but no in-text citation
- **0** Cited and Quoted 0%
  Matches with in-text citation present, but no quotation marks

Top Sources

- 3% Internet sources
- 1% Publications
- 8% Submitted works (Student Papers)

Top Sources

The sources with the highest number of matches within the submission. Overlapping sources will not be displayed.

1	Student papers	
Universiti Teknologi Malaysia on 2025-06-14		4%
2	Student papers	
Universiti Teknologi Malaysia on 2024-06-16		2%
3	Student papers	
Universiti Teknologi Malaysia on 2024-07-04		1%
4	Student papers	
Universiti Teknologi Malaysia on 2026-06-25		<1%
5	Internet	
d-nb.info		<1%
6	Publication	
Leonarda Pušić, Tomislav Jagušt, Marko Horvat, Bartol Boras. "The Impact of a Hi...		<1%
7	Student papers	
Universiti Teknologi Malaysia on 2024-07-04		<1%
8	Internet	
repositories.lib.utexas.edu		<1%
9	Student papers	
Asia Pacific University College of Technology and Innovation (UCTI) on 2026-03-20		<1%

CHAPTER 5

RESULTS, TESTING AND DISCUSSION

5.1 Introduction

This chapter presents the implementation outcomes of the Al-Muneer Online Portal after transitioning from the PSM1 design phase into the PSM2 development phase. The system was built using the Spring Boot framework with a PostgreSQL database, secured by JWT-based authentication, and rendered through server-side Thymeleaf templates enhanced with vanilla JavaScript and Chart.js. The implementation followed the Waterfall model established in Chapter 3, with the requirements and design artefacts from Chapters 3 and 4 serving as the blueprint for coding, interface construction, and testing.

The discussion is organised into three main areas. Section 5.2 documents the coding of the system's main functions, aligned with the changelog of implemented versions from v0.3 to v0.7. Section 5.3 presents the essential user interfaces that demonstrate the system's results and achievements, accompanied by explanatory placeholders for screenshots. Section 5.4 explains the testing activities carried out, covering black box, white box, and user testing approaches. The chapter concludes with a summary in Section 5.5.

5.2 Coding of System's Main Functions

The backend of the Al-Muneer Online Portal was developed in Java 21 using Spring Boot 3.4.4, with Maven as the build tool. PostgreSQL 16 was chosen as the relational database, and Spring Security with JWT tokens and BCrypt hashing was used to protect the administrator panel. The frontend was implemented with HTML5, CSS3, Thymeleaf, and vanilla JavaScript, supported by Chart.js for data visualisation. This section explains the key functional modules that were coded, following the iterative changelog from v0.3 through v0.7.

5.2.1 Backend Foundation and Security

The backend follows a layered architecture comprising controllers, services, repositories, and JPA entities. The `AdminAuthController` handles login using Spring Security, while JWT tokens stored in HTTP-only cookies maintain authenticated sessions. Passwords are hashed with BCrypt, and a `DataSeeder` creates a default administrator account on the first run. Domain entities include `BookingInquiry`, `PaymentProof`, `AvailabilitySlot`, `PricingTier`, `MediaItem`, `Feedback`, `PageVisit`, and `NotificationTemplate`, all mapped to PostgreSQL tables through Spring Data JPA.

5.2.2 Visitor Booking Flow and Reference Code System

A central achievement of the implementation is the unified visitor booking flow. The home page was converted into a single-page scrollable landing page with anchor navigation, consolidating the hero section, venue information, media gallery, availability calendar, and pricing packages. Visitors interact with an interactive calendar where future dates default to Available and past dates are dimmed. Tapping an available date reveals a single "Submit inquiry" call-to-action that passes the selected date to `/inquiry?date=YYYY-MM-DD`. Similarly, pricing package cards include "Book [package]" buttons that pre-select the tier via `/inquiry?pricingId=...`; if a date was already chosen, both parameters are combined.

The inquiry landing page unifies two capabilities: submitting a new inquiry and looking up an existing inquiry by a visitor-facing reference code. Each inquiry is assigned a random 9-digit `referenceCode` stored in a 150-day HTTP-only cookie named `inq_ref`. Visitors can also cancel their own inquiry directly from the confirmation page, provided no payment proof has been attached, which frees the associated availability slot.

5.2.3 Payment Proof, Status Cascade, and Notifications

The payment proof module allows visitors to upload offline payment receipts as image files. The `FileUploadUtil` stores files under `uploads/proofs/` and records the UUID filename in the `PaymentProof` entity. When an administrator verifies a proof, an atomic status cascade is triggered: the payment proof status becomes Verified, the linked `BookingInquiry` moves to CONFIRMED, and the associated `AvailabilitySlot` is marked BOOKED.

Dynamic WhatsApp notifications are implemented through database-backed templates. Administrators can create, edit, and delete notification templates from the admin panel. On the inquiry or payment detail page, a dropdown lets the admin choose a template, preview the message with placeholders resolved, and generate a `wa.me` deep-link client-side. The configured country code (`app.country.code=+967`) is prepended to normalised WhatsApp numbers.

5.2.4 Administrator Modules and Analytics

The admin dashboard provides a daily workload overview, including counts of new and active inquiries, pending payment proofs, unreviewed feedback, visits today and over the last seven days, average feedback rating, recent inquiries, and top pages. The analytics page visualises traffic data with Chart.js, offering a bar chart of top pages and a line chart of daily trends over the last 30 days. The reports page presents inquiry status, payment status, and feedback rating distributions using pie and bar charts, with optional date range filtering via `?fromDate=` and `?toDate=` query parameters.

5.2.5 AI Integration with Gemini

Version v0.7 introduced three non-blocking AI advisors powered by the Google GenAI Java SDK (`google-genai` 1.56.0) and the Gemini model:

- **AI Business Report Advisor** derives booking conversion and cancellation rates and prompts Gemini for three number-backed action bullets.
- **AI Feedback Advisor** separates low-rating complaints from positive highlights and recommends what the owner should address first.
- **AI Traffic Funnel Advisor** maps Home → Gallery → Pricing → Inquiry visit counts to identify funnel drop-off and suggest a concrete conversion improvement.

All AI panels load asynchronously through dedicated `GET /ai-insight` endpoints after the page renders, ensuring that the main interface remains fast and degrades gracefully if the API is unavailable.

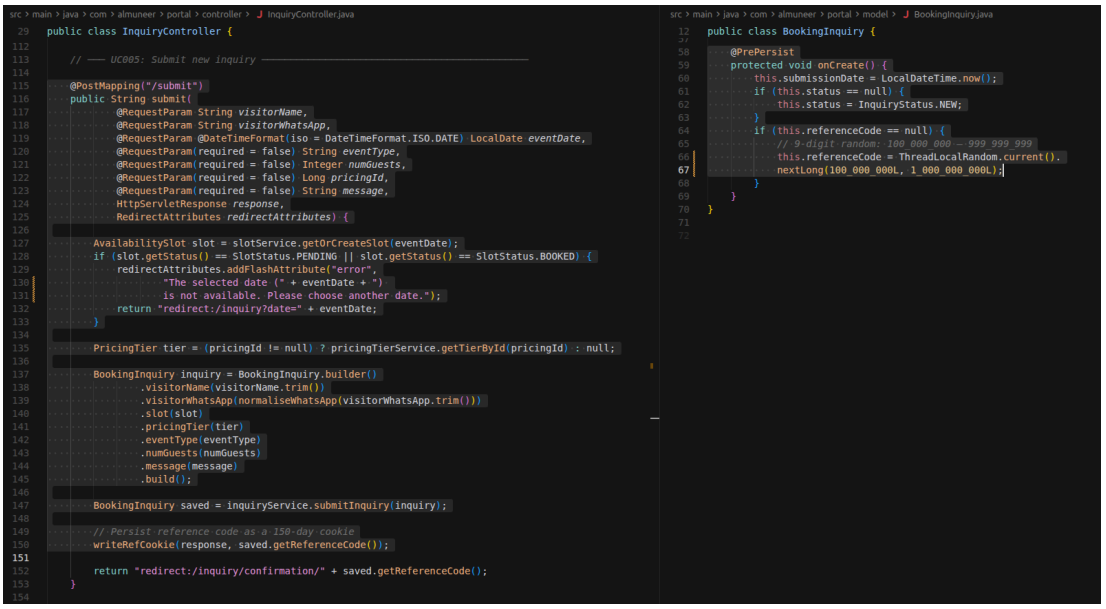

Figure 5.1: Code Snippet: Booking Inquiry Submission Flow

5.3 Essential Interfaces Showing System's Results and Achievements

This section describes the key interfaces that demonstrate the functional results of the implementation. Screenshots of the actual system are included as placeholders for the final report.

5.3.1 Visitor Home Page and Booking Calendar

The visitor home page presents a single-page scroll experience. The hero section is followed by venue information with an embedded Google Maps iframe, a masonry media gallery with label-based filters, an interactive availability calendar, and pricing packages. Anchor navigation and scroll-spy highlight the active section. The calendar supports month navigation, visually distinguishes available and booked dates, and reveals a contextual "Submit inquiry" action when a visitor selects an available day.

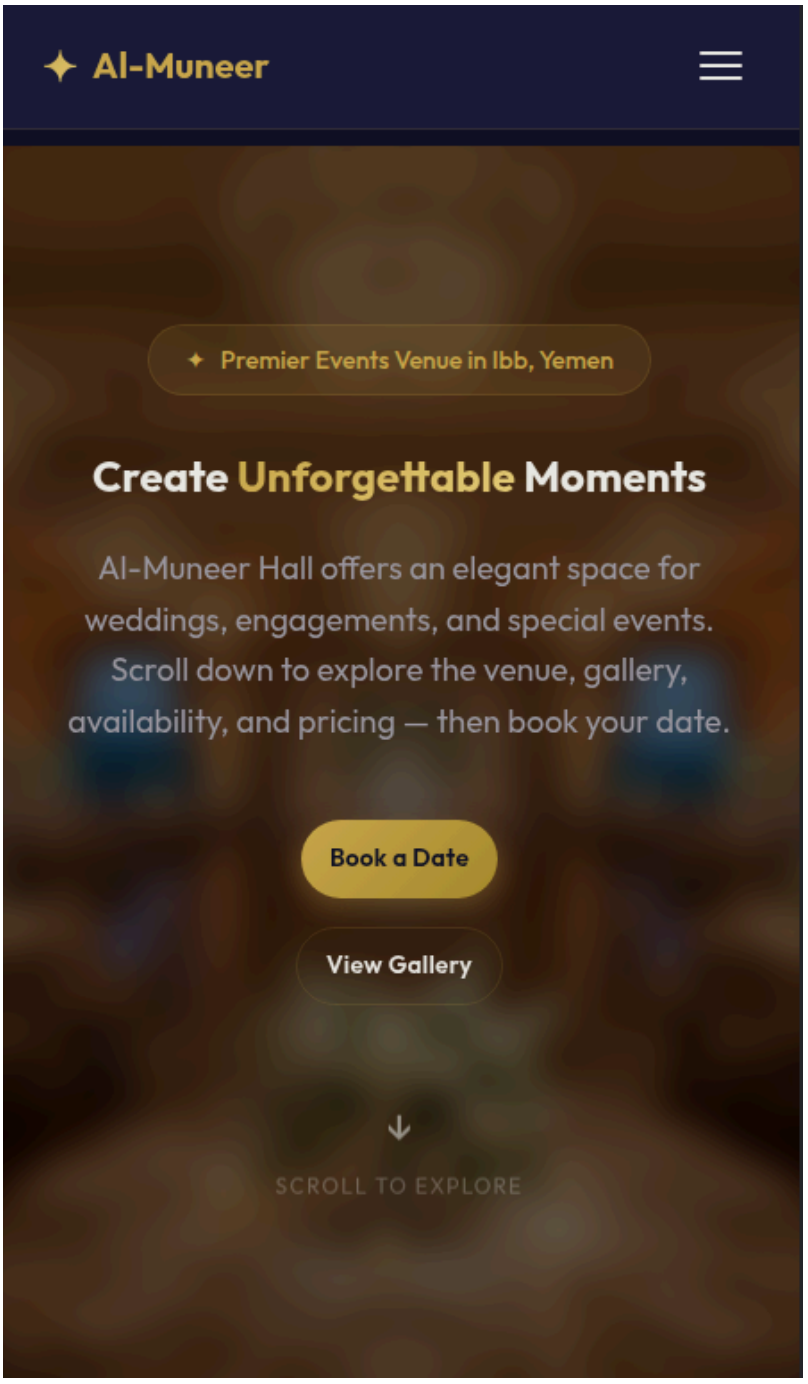

Figure 5.2: Visitor Home Page with Single-Page Scroll Layout

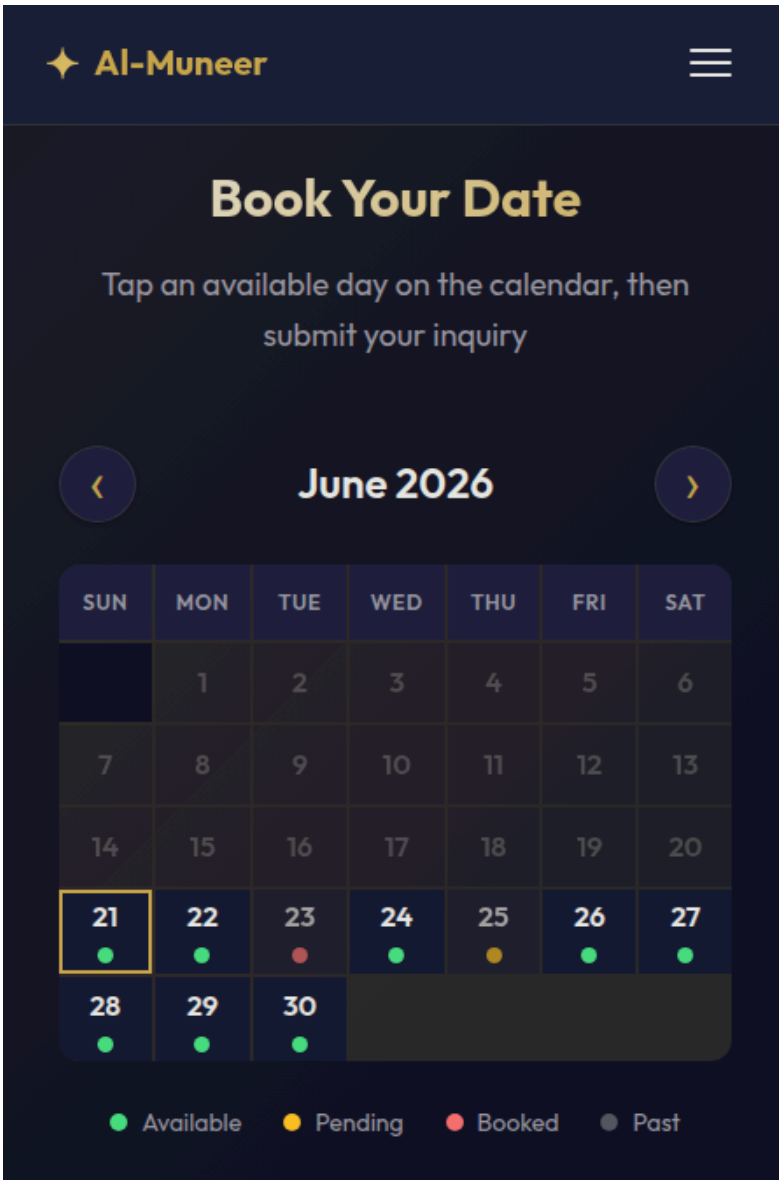

Figure 5.3: Visitor Availability Calendar and Inquiry CTA

5.3.2 Inquiry Form and Reference Code Lookup

The `/inquiry` page serves both new submissions and existing-lookup use cases. When arriving from the home page, the date and/or pricing package are pre-filled in the form. After submission, the visitor sees a confirmation page with the 9-digit reference code and a self-cancellation option. The same page contains a lookup field where returning visitors can enter their reference code to retrieve inquiry status.

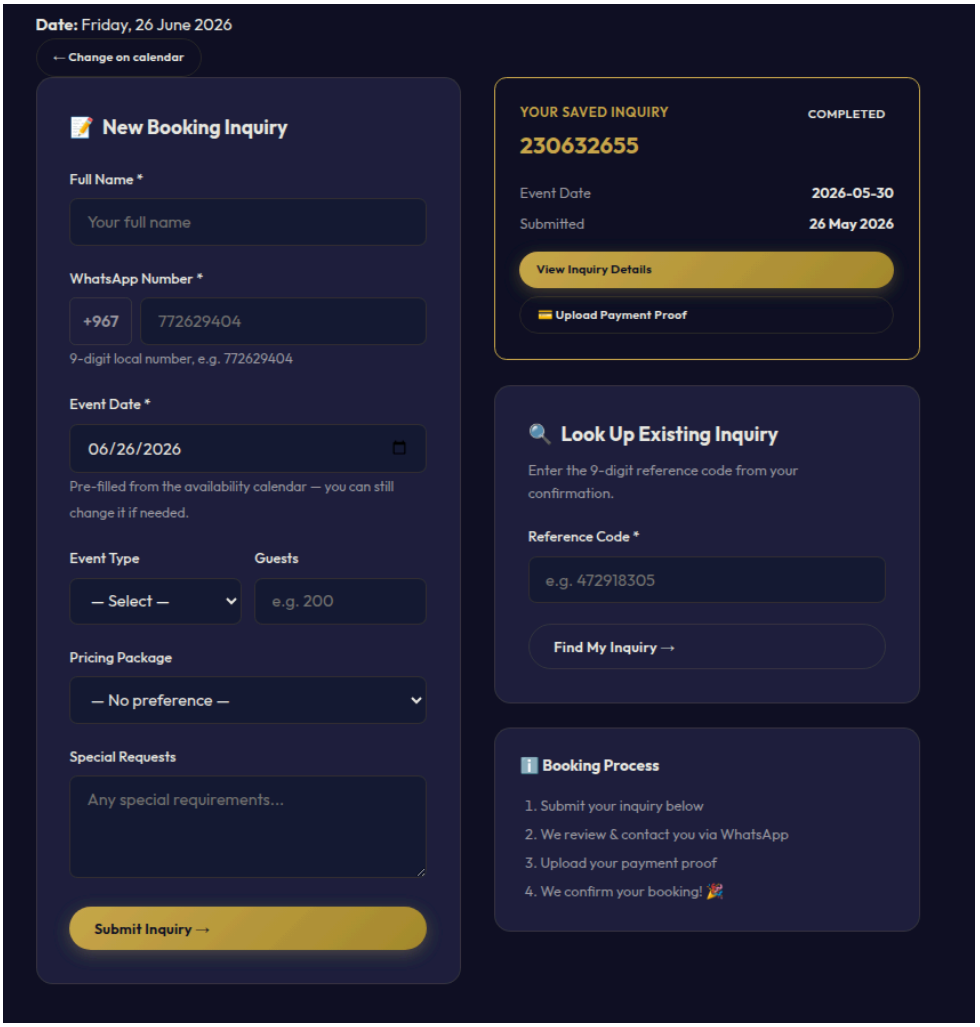

Figure 5.4: Inquiry Form with Pre-filled Date and Package

5.3.3 Admin Dashboard and Analytics

The administrator dashboard gives the owner an at-a-glance view of daily operations. Cards display new and active inquiries, pending proofs, unreviewed feedback, and recent traffic. The analytics page extends this with interactive Chart.js visualisations and the asynchronous AI Traffic Funnel Advisor panel.

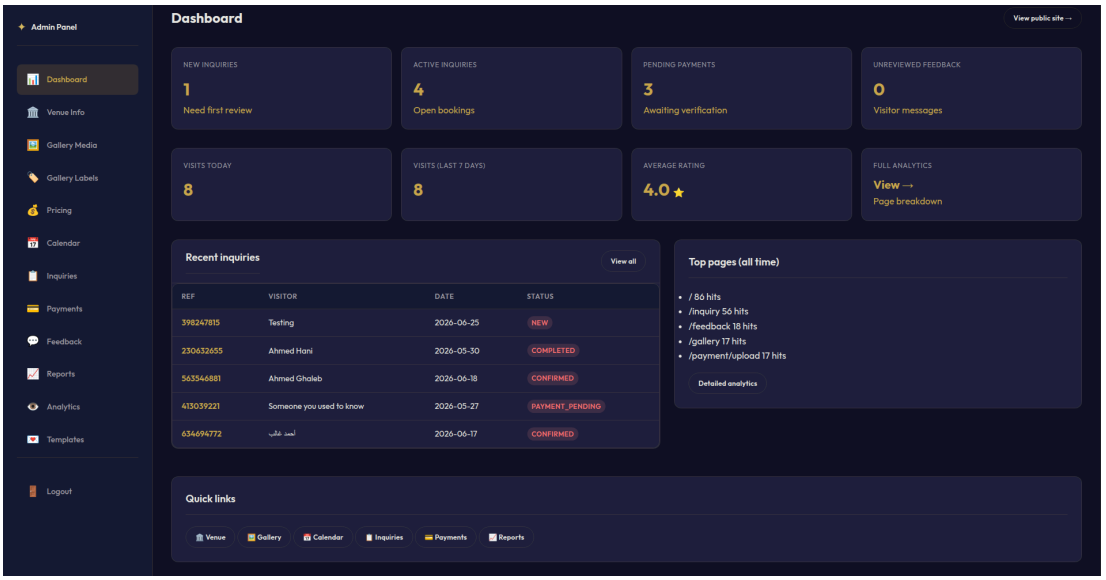

Figure 5.5: Administrator Dashboard Overview

5.3.4 Admin Inquiry and Payment Management

The inquiry management panel includes status filter chips with per-status counts, client-side search, and an "active only" default view. Each row shows the visitor reference code, contact details, event date, and current status. From the detail view, the administrator can update status, view linked payment proofs filtered by inquiry, and send a dynamic WhatsApp message using a template selector. The payment verification screen displays the uploaded receipt image and supports the verification cascade described in Section 5.2.3.

Figure 5.6: Administrator Inquiry Management Panel

5.3.5 Reports with Visualisations

The reports page summarises operational data through visual charts. Pie charts show inquiry status and payment status breakdowns, while a bar chart displays feedback rating distribution. Date range filters allow the owner to analyse trends over specific periods. The AI Business Report Advisor panel loads asynchronously to provide actionable insights based on the current report data.

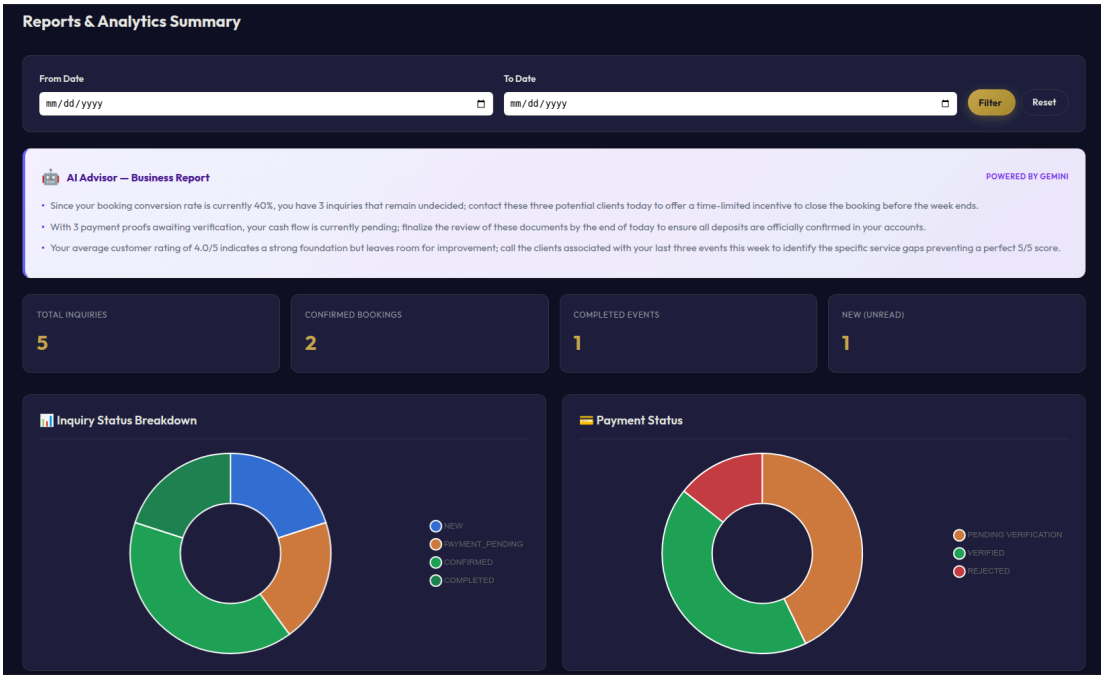

Figure 5.7: Reports Page with Visual Charts

Figure 5.8: Analytics Dashboard with Chart.js Visualizations

5.4 Testing

Testing was conducted throughout the implementation phase to verify that the Al-Muneer Online Portal meets the functional and non-functional requirements defined in the SRS. The approach combined black box, white box, and user testing.

5.4.1 Black Box Testing

Black box testing focused on system flows, input/output behaviour, and error messages without examining internal code. Test scenarios were derived directly from the use cases in the SRS and the changelog features:

- **Visitor flow:** Submitting an inquiry with valid and invalid data, pre-filling date/package from the home page, looking up an inquiry by reference code, and self-cancelling an inquiry.
- **Payment proof flow:** Uploading valid and invalid file types, verifying a proof as an admin, and confirming the status cascade to inquiry and slot.

- **Admin flow:** Logging in, managing venue info and gallery labels, updating calendar slots, filtering inquiries, sending WhatsApp messages from templates, and generating reports.
- Error handling: Invalid login credentials, missing required fields, oversized uploads, and unauthorized access attempts all produced clear user-friendly messages.

The black box tests confirmed that the system behaves according to specification and handles edge cases gracefully.

5.4.2 White Box Testing

White box testing examined internal logic and code paths. Unit tests and integration tests were performed on the following components:

- **Reference code generation:** Verified that `BookingInquiry` receives a unique 9-digit code.
- **WhatsApp normalisation:** Confirmed that `normaliseWhatsApp()` strips non-digits and prepends the configured country code.
- **Status cascade:** Validated that verifying a `PaymentProof` updates the linked `BookingInquiry` to CONFIRMED and the `AvailabilitySlot` to BOOKED.
- **JWT security:** Verified token creation, validation, and expiration handling.

AI service fallback: Confirmed that `GeminiService` returns a graceful fallback message on API failure without crashing the page.

These tests ensured that critical business rules and security mechanisms operate correctly at the code level.

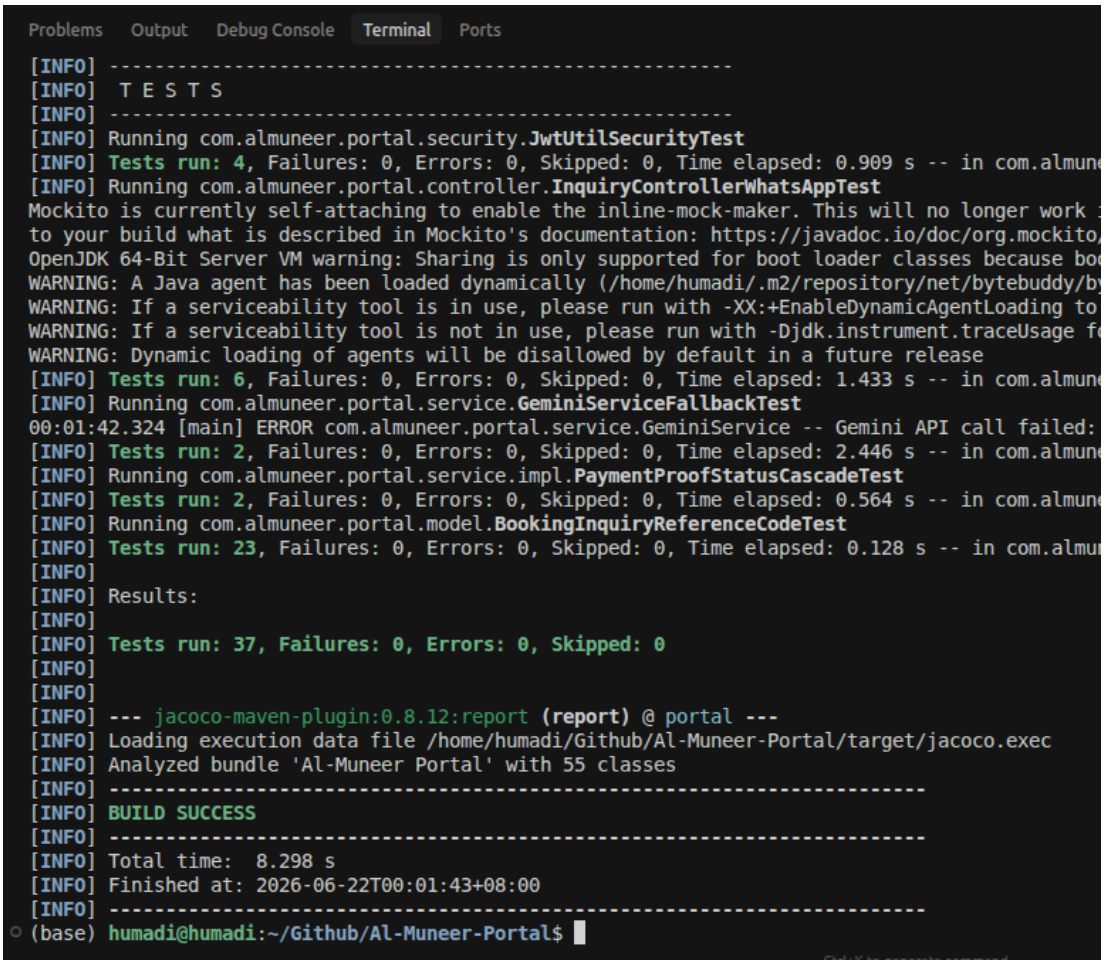

Figure 5.9: Unit Testing Log

5.4.3 User Testing

User testing was conducted with the project stakeholder, Mr. Ahmed Almunajid, the owner of Al-Muneer Hall. The owner performed representative tasks for the English version of the website, including updating venue information, uploading gallery images, checking new inquiries, verifying a payment proof, and viewing reports. Feedback was collected on interface clarity, workflow efficiency, and the usefulness of the AI advisors. Minor adjustments to dashboard card ordering, and notification template defaults were made based on this feedback.

5.5 Chapter Summary

This chapter presented the implementation results of the Al-Muneer Online Portal. Section 5.2 described the coding of main functions, including the Spring Boot backend, visitor booking flow with reference codes, payment proof verification cascade, dynamic WhatsApp notifications, admin dashboard, analytics, reports, and Gemini AI advisors. Section 5.3 presented the essential interfaces that demonstrate these achievements, and Section 5.4 explained the black box, white box, and user testing performed. The system was found to satisfy the requirements specified in the SRS and to provide a practical, user-friendly solution for both visitors and the hall owner.